

AI-Based ECG Diagnostic Assistant

Web build plan: phases from backend to hardware integration

Plan written 3 Oct 2026. Phases 0 to 7 are pure software and need no hardware. ESP32 and real sensor data come in **November** (Phase 8), after the whole web stack already works on simulated data.

1. Guiding rules

- **Build inside-out.** Prove each layer before connecting the next: database, then APIs, then dashboard, then simulated data, then real hardware.
- **One contract, many senders.** The simulator and the ESP32 send the same messages to the same FastAPI endpoints. When the ESP32 arrives, nothing server-side changes.
- **Wi-Fi, not Bluetooth.** The ESP32 posts straight to FastAPI over Wi-Fi. No gateway process.
- **One owner per table.** Django owns identity and business data. FastAPI owns telemetry. No cross-service foreign keys.
- **Understand every layer.** Each phase ends with you able to explain it in an interview. Have the agent explain as it builds, and write a short note per phase in docs/.
- **Plan first, then implement.** For each phase, ask the agent for a plan, review it, then say go.

2. Architecture and ownership

	Django (identity + business)	FastAPI (telemetry + compute)
Owns	Users, roles, patients, doctors, devices, sessions, reports, Django Admin	Sensor readings, ECG chunks, predictions, live streaming
Endpoints	Auth, patient/device/session CRUD, report generation	/sensor/connect, /sensor/data, /sensor/status, WS /ws/sensor, /inference
Schema	core (Django migrations)	telemetry (SQLAlchemy + Alembic)
Auth	Issues user JWTs (SimpleJWT) and signed device tokens	Verifies tokens locally, reads role from claims

React talks to both: Django for login, patients, devices and reports; FastAPI for the live stream and readings. One PostgreSQL instance serves both schemas. Telemetry rows reference device_id and session_id as plain UUIDs.

Major Project/	
backend/django/	accounts, patients, devices, sessions, reports
backend/fastapi/	ingestion, validation, DSP, inference, streaming
frontend/	React dashboard
simulator/	fake + replayed sensor data (same contract as ESP32)
ml/	datasets, training, evaluation, ONNX export
docker/	compose files
docs/	architecture, API contract, per-phase notes

3. Timeline at a glance

Phase	What	Target
0	Repo, Postgres in Compose, CI skeleton	3 - 6 Oct
1	Django core: auth, roles, patients, devices, sessions	7 - 14 Oct
2	FastAPI telemetry + ingestion contract + JWT trust	15 - 21 Oct
3	Simulator: scalar sensors + ECG replay	22 - 25 Oct
4	React dashboard with live graphs	22 Oct - 2 Nov

Phase	What	Target
5	Signal processing + ML inference slot	parallel, Oct - Nov
6	Reports, permissions, admin polish	early Nov
7	Hardening, tests, demo on simulated data	early Nov
8	ESP32 integration: one sensor, then ECG, then the rest	November
9	Docker images, CI/CD, AWS, Terraform, monitoring	after Phase 8 works

Dates are a suggestion. If something slips, move the date, not the order.

4. Phase details

PHASE 0

Setup

3 - 6 Oct

Goal: A clean workspace where every later piece lands in a pipeline that already runs.

- Create the repo and folder layout above; git, branch rules, .gitignore, .env.example
- Docker Compose with PostgreSQL only (create schemas core and telemetry)
- Empty Django project and empty FastAPI app, each connecting to Postgres
- GitHub Actions: lint (ruff) and a placeholder test on every push
- Ask the agent for an architecture proposal first; review before implementing

Done when: docker compose up starts Postgres, both apps connect, CI is green.

PHASE 1

Django core

7 - 14 Oct

Goal: Everything about who exists and what they may do.

- Custom user model with roles: patient, doctor, admin
- Patient and Doctor profiles; doctor-to-patient assignment
- Device model (name, owner, status) and device registration producing a signed device token
- Session model (patient, device, start, end, status), created by an authenticated user
- JWT login/refresh (SimpleJWT); permission classes per role
- Django Admin configured for all core models; pytest tests for permissions

Done when: A doctor can log in, see only assigned patients, register a device and start a session via API. Admin works.

PHASE 2

FastAPI telemetry

15 - 21 Oct

Goal: Accept, validate and store sensor data, and trust Django's tokens.

- SQLAlchemy models + Alembic migrations in the telemetry schema
- Ingestion contract (write it down in docs/ first): POST /sensor/data for scalar readings; chunked ECG upload with session_id, start time, sample rate, samples, chunk index (idempotent)
- POST /sensor/connect and GET /sensor/status (device heartbeat, last seen)
- Verify user JWTs and device tokens locally; reject data for unknown sessions
- Pydantic validation (ranges, units, timestamps); reject malformed data cleanly
- WS /ws/sensor: broadcast new readings to subscribed viewers
- pytest tests for ingestion, auth failures and duplicate chunks

Done when: A curl call with a device token stores a temperature reading; a bad token or bad value is rejected; a WebSocket client sees the reading live.

PHASE 3	Simulator	22 - 25 Oct
Goal: Realistic data without hardware, speaking the exact contract the ESP32 will use. • Python script that registers/uses a device token and posts temperature, SpO₂, pulse and motion readings on a timer • ECG mode: replay MIT-BIH or CinC 2017 records, resampled to 200 Hz, as 1 to 5 second chunks • Options for noise, dropped chunks and device going offline, to test edge cases • ECG storage as chunks (array of samples per row), not one row per sample Done when: Running the simulator fills the database with all four sensor types and ECG chunks for a session.		
PHASE 4	React dashboard	22 Oct - 2 Nov
Goal: A working app you can demo with no physical sensor attached. • Login, role-aware navigation (patient / doctor / admin) • Patient list, device list, start/stop session • Live monitor: ECG waveform (uPlot or similar), heart rate, SpO₂, temperature, motion, over WS /ws/sensor • Session history and a waveform viewer for past recordings • Device status badge (online/offline from heartbeat) and a visible screening-aid disclaimer on every screen Done when: Start the simulator, watch live graphs update in the browser, then find that session in history.		
PHASE 5	Signal processing + ML slot	parallel, Oct - Nov
Goal: Deterministic ECG analysis first, ML plugged in behind a stable interface. • Shared DSP module: 50 Hz notch, 0.5 to 40 Hz band-pass, R-peak detection, BPM, signal-quality check • Test the detector against MIT-BIH annotations • ML in ml/: CinC 2017 four-class model (Normal / AFib / Other / Noisy), patient-wise split, export to ONNX • FastAPI /inference endpoint that loads the ONNX model; predictions stored with model version and probability • Noisy signals are marked 'cannot read', never guessed • Slot is left for a teammate or later work if ML is not your part Done when: A replayed ECG session produces BPM and a stored prediction with probability, shown on the dashboard.		
PHASE 6	Reports and permissions	early Nov
Goal: The Django-side features that make the project look like a real product. • Session report (summary stats, predictions, disclaimer) generated by Django from telemetry data, exportable as PDF • Doctor view: assigned patients, flagged sessions, notes • Admin dashboard: devices online, sessions per day, recent flags • Audit-style logging of who viewed or exported what Done when: A doctor opens a patient's session and downloads a PDF report; a different doctor cannot.		

Goal: A finished web app before any hardware risk is added.

- End-to-end test: login, session, simulator, live view, report
- README with setup steps and an architecture diagram; API docs (OpenAPI) up to date
- Short screen-recorded demo video using simulated data
- Freeze the ingestion contract; list exactly what the ESP32 must send

Done when: A fresh clone runs the full system with one command and the demo works.

Goal: Replace the simulator with the real device, one sensor at a time.

- ESP32 connects to Wi-Fi (use a phone hotspot if college Wi-Fi isolates clients); point it at the laptop's local IP
- Step 1: one scalar sensor (e.g. MLX90614 temperature) posting readings with the device token
- Step 2: add MAX30102 (SpO₂/pulse), then MPU6050 on the shared I2C bus
- Step 3: ECG via AD8232 at 200 Hz, batched chunks; leads-off handling; verify no dropped chunks
- Validate BPM against a reference (pulse oximeter or chest strap)
- Safety: battery or power bank only while electrodes are attached; consenting volunteers only; keep records anonymised
- Compare real signals to public datasets; add real noise to training data; keep your own recordings out of the public test split

Done when: Live ECG and sensor values from the real device appear on the same dashboard, with no server changes.

Goal: Only package and deploy a system that already works.

- Dockerfiles for Django, FastAPI and React; production Compose file
- GitHub Actions: test, build, scan, push images
- AWS: start simple (one EC2 with Compose), then RDS, S3, ALB/TLS; Terraform if time allows
- Monitoring: /health and /metrics, Grafana dashboard, device heartbeat alert, cost budget
- ESP32 switches from a LAN IP to the domain name (needs TLS certificate handling on the device)

Done when: A merge to main deploys to staging automatically and the ESP32 reaches the cloud server.

5. Starter prompt for the coding agent (Phase 0)

Before writing application code, inspect the existing project and propose an architecture for a multi-sensor ESP32 healthcare monitoring system: Django (accounts, roles, devices, sessions, reports), FastAPI (sensor ingestion, validation, WebSocket streaming, signal processing, ML inference), PostgreSQL (separate core and telemetry schemas), and React. The ESP32 sends data over Wi-Fi via HTTP/WebSocket; no Bluetooth. A simulator will use the same contract before hardware exists. Do not implement yet. Explain each service's responsibilities, the auth trust boundary (Django issues JWTs, FastAPI verifies), the data model, and the directory structure. Keep it simple and explain the reasoning so I can defend it in interviews.

Note: this system is a screening aid and not a medical diagnosis. Keep that wording in the UI, reports and README.